

Assignment Part 1 — Scheduling Platform Backend: Foundations and CRUD

1. Objective

Build the foundation of a scheduling platform backend. A **host** registers, publishes bookable

event types (e.g. “30 Minute Intro Call”), and declares when they are available: recurring weekly *availability rules* plus per-date *exceptions* for holidays, one-off blocks, and extra hours. An unauthenticated public endpoint exposes a host’s event type so an invitee-facing client can render a booking page.

Part 1 covers everything up to — but not including — turning that availability into concrete

bookable time slots. There are no background jobs, no third-party integrations, and no concurrency-sensitive writes in this part. Every endpoint is a straightforward request validate persist respond cycle.

This part assesses your ability to:

- Structure a layered FastAPI service with strict separation of concerns.
- Model a non-trivial relational domain and evolve it with migrations.
- Design a consistent, well-documented HTTP contract with disciplined error handling.
- Write validation that lives in the schema layer, not scattered through business logic.

2. Scope

In scope

Area	Detail
------	--------

Project scaffold	Layout, configuration, dependency management
Database	PostgreSQL schema for users, event types, availability rules, availability exceptions
Migrations	Alembic chain producing that schema from empty
HTTP contract	Response envelope, error envelope, status code policy, OpenAPI docs
CRUD APIs	Users, event types, availability rules, availability exceptions
Public API	Single unauthenticated event-type lookup

Out of scope (Part 2)

Slot generation, the `slots` and `bookings` tables, the booking endpoint and its concurrency handling, Temporal workflows and the worker process, Google Calendar and OAuth, and outbound email. Do not build them, and do not add speculative abstractions in anticipation of them.

Part 2 extends the structure you deliver here, so the layering you establish now matters more than any placeholder.

Also deliberately out of scope: authentication, CORS, rate limiting, request-logging middleware, API versioning prefixes, containerisation, and automated tests.

3. Technology Stack

The stack is fixed. Substitutions require written approval.

Concern	Required
Language	Python 3.12+ , full type hints throughout
Web framework	FastAPI + Uvicorn (ASGI)
Validation / serialisation	Pydantic v2
Database	PostgreSQL 16

ORM	SQLAlchemy 2.0 , async, <code>asyncpg</code> driver
Migrations	Alembic
Date / timezone	zoneinfo (stdlib) + pendulum
Configuration	pydantic-settings
Package manager	uv (or Poetry) with a committed lockfile

`psycopg2` is forbidden — this service is async end to end.

4. Deliverables

1. Git repository with clean, incremental commit history.
 2. `README.md` : architecture overview, how to run PostgreSQL, how to run the API.
 3. `pyproject.toml` with a committed lockfile.
 4. Alembic migration chain that builds the §7 schema from an empty database.
 5. `.env.example` listing every variable from §11, with safe defaults.
 6. OpenAPI docs at `/docs` , with every request and response schema accurately described.
 7. `docs/DECISIONS.md` : a short record of design decisions, trade-offs, and known gaps.
-

5. Required Project Structure

Layering is enforced at review. Dependencies point **inward and downward only**:

`router` `service` `repository` `database` .

- A router must never import a repository or construct a SQLAlchemy query.
- A repository must never import a service and must contain no business rules.
- A service must contain no SQL and no HTTP concepts (no `Request` , no `status_code`).

```

app/
  main.py          # app factory: middleware, routers, exception handlers
  config/
    settings.py    # pydantic-settings; a single Settings object
    database.py    # async engine, session factory, get_session dependency
  api/

```

```
deps.py          # shared dependencies (host identity, session)
routers/
  health.py
  users.py
  event_types.py
  availability.py
  public_events.py
schemas/         # Pydantic request/response models
  common.py      # ApiResponse[T], error models
  user.py
  event_type.py
  availability.py
models/          # SQLAlchemy declarative models
repositories/    # Data access only
services/        # Business rules only
  user_service.py
  event_type_service.py
  availability_service.py
errors.py        # ApiError hierarchy + factory helpers
utils/
  slug.py
alembic/
```

Part 2 will add `app/temporal/` , `app/mailer/` , and further services into this same tree. Leave room for them; do not stub them.

6. Terminology: What Each Layer Refers To

These four words get used interchangeably in casual conversation and mean four distinct things

here. The layering rules above only make sense once the distinction is clear.

Model

A **model** is a SQLAlchemy declarative class in `app/models/` that maps one-to-one onto a database table. It is the single place the physical schema is described in Python: columns, types, defaults, constraints, indexes, and relationships to other models. One model class describes one table; one model **instance** represents one row of that table.

```
class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(name="email", unique=True)
    timezone: Mapped[str] = mapped_column(name="timezone", server_default="UTC")

    event_types: Mapped[list["EventType"]] = relationship(back_populates="host")
```

Two rules govern models in this project:

1. **A model is never returned from a router.** Serialising a model directly leaks internal column names and any field you later add — including ones that should stay private — into the public API, and it triggers lazy-loading of relationships at serialisation time, often outside the session, which either fails or fires surprise queries. Convert to a Pydantic schema first.
2. **Models describe storage, not behaviour.** Business rules do not live on the model class. A method like `user.can_book(slot)` belongs in a service.

Models are also what Alembic compares against to generate migrations, so a change to a model without a corresponding migration means your code and your database have silently diverged.

Schema

A **schema** is a Pydantic class in `app/schemas/`. It defines the shape of JSON crossing the HTTP boundary — one for the request body, one for the response — along with every validation rule: field types, length bounds, regexes, enum membership, and cross-field constraints.

A schema is a *contract*; a model is *storage*. They deliberately differ. `CreateUserRequest` has no `id` (the database assigns it) and no `createdAt`. `PublicEventTypeResponse` exposes five fields out of a twelve-column table. Collapsing the two into one class couples your API

surface to your table layout, so that every schema migration becomes a breaking API change.

Repository

A **repository** in `app/repositories/` is the only place SQL or SQLAlchemy query construction is allowed. It takes plain arguments and a session, returns models or scalars, and makes no decisions: no validation, no permission checks, no error raising beyond “not found”. If a function reads like a sentence about the business, it is in the wrong file.

Service

A **service** in `app/services/` holds the business rules: what a valid state transition is, what makes a request a conflict, what has to happen and in what order. It orchestrates one or more repository calls, and it raises the domain errors that §8.2 maps onto status codes. It must be readable without knowing whether the storage underneath is PostgreSQL or anything else — hence no SQL, and no HTTP.

7. Data Model

PostgreSQL. Table names are snake_case plural; column names are camelCase. Use `mapped_column(name="camelCase")` so Python attributes stay snake_case while the database and JSON surface use camelCase.

Every table carries:

- `createdAt TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP`
- `updatedAt TIMESTAMP(3) NOT NULL` , maintained by the application
(`onupdate=func.now()`)

users

Column	Type	Constraints
id	SERIAL	PK

email	TEXT	NOT NULL , UNIQUE
name	TEXT	NOT NULL
slug	TEXT	NOT NULL , UNIQUE
timezone	TEXT	NOT NULL DEFAULT 'UTC' , IANA identifier

event_types

Column	Type	Constraints
id	SERIAL	PK
hostId	INTEGER	FK users.id ON DELETE CASCADE
title	TEXT	NOT NULL
description	TEXT	nullable
slug	TEXT	NOT NULL
durationMinutes	INTEGER	NOT NULL
isActive	BOOLEAN	NOT NULL DEFAULT true
locationType	TEXT	NOT NULL DEFAULT 'online'
locationValue	TEXT	nullable
bufferBeforeMinutes	INTEGER	NOT NULL DEFAULT 0
bufferAfterMinutes	INTEGER	NOT NULL DEFAULT 0

Constraint: UNIQUE (hostId, slug) . Two hosts may both own 30-minute-meeting ; one host may not own it twice.

The buffer columns are unused in Part 1 — they describe padding around a meeting and are consumed by slot generation in Part 2. Persist and validate them now.

availability_rules

Recurring weekly availability.

Column	Type	Constraints
id	SERIAL	PK
userId	INTEGER	FK users.id ON DELETE CASCADE
weekday	INTEGER	NOT NULL , 0 = Sunday ... 6 = Saturday
startTime	TEXT	NOT NULL , HH:mm 24-hour
endTime	TEXT	NOT NULL , HH:mm 24-hour
isActive	BOOLEAN	NOT NULL DEFAULT true
timezone	TEXT	NOT NULL DEFAULT 'UTC'

Index: (userId, weekday) .

The weekday encoding is 0 = Sunday, which matches neither Python's date.weekday()

(0 = Monday) nor ISO weekday (1 = Monday ... 7 = Sunday). Isolate the conversion in one helper.

Part 2 depends on getting this right.

Times are stored as HH:mm text alongside an IANA timezone, not as TIME or as UTC offsets.

A rule means “09:00 to 17:00 local wall-clock time in this zone”, which is a different instant

before and after a daylight-saving transition. Resolving that is Part 2’s problem; storing it faithfully is Part 1’s.

availability_exceptions

Per-date overrides that layer on top of the weekly rules.

Column	Type	Constraints
id	SERIAL	PK
userId	INTEGER	FK users.id ON DELETE CASCADE
date	DATE	NOT NULL
type	TEXT	NOT NULL , one of BLOCK_FULL_DAY BLOCK_PARTIAL ADD_AVAILABLE_WINDOW

startTime	TEXT	nullable, HH:mm
endTime	TEXT	nullable, HH:mm
timezone	TEXT	NOT NULL DEFAULT 'UTC'
reason	TEXT	nullable

Index: (userId, date) .

Enum policy

Status and type columns are TEXT , not PostgreSQL enums — adding a value to a PG enum requires

a migration and locks in production. Model them as Python StrEnum at the application boundary

and validate on the way in, so the constraint is enforced by Pydantic and visible in OpenAPI.

8. Cross-Cutting HTTP Contract

This section defines behaviour Part 2 builds on unchanged. Get it right once, centrally.

8.1 Response envelope

Every business endpoint returns:

```
{ "success": true, "data": { }, "message": "optional human-readable string" }
```

Implement a generic ApiResponse[T] Pydantic model and declare it as the response_model so

OpenAPI reflects the real shape. Do not hand-assemble dicts in routers, and do not return bare model instances.

8.2 Error envelope

```
{ "success": false, "message": "Validation failed", "details": [ ] }
```

- `details` carries structured validation errors, or a traceback when `ENVIRONMENT=development`. A traceback must never escape a non-development environment.
- Register three handlers in the app factory:
 1. Your `ApiError` hierarchy its own status code and message.
 2. `RequestValidationError` **rewrite FastAPI's default 422 into this envelope with status 400.**
 3. A catch-all `Exception` 500, logged with the traceback, generic message in the body.
- FastAPI's default `{"detail": ...}` shape must not appear in any response, including 404s for unmatched routes.

8.3 Status codes

Code	Used for
200	Successful <code>GET</code> , <code>PATCH</code> , <code>DELETE</code>
201	Successful <code>POST</code> create
400	Validation failure; malformed or missing <code>x-user-id</code>
404	Entity not found; unmatched route
409	Duplicate email, or event-type slug already taken for that host
500	Unhandled error

8.4 Identifying the host

There is no authentication in this assignment. Endpoints that act on a host's own data simply

read the host's integer ID from an `x-user-id` request header:

```
x-user-id: 42
```

Implement it as a small FastAPI dependency returning `int`, applied at router level for `/api/event-types` and `/api/availability`. A missing or non-numeric header is a **400**.

Host-scoped endpoints operate only on rows belonging to that ID. A request for a row owned by someone else is a **404**, exactly as if it did not exist.

`/api/users` , `/api/public/*` , and `/health` require no header at all.

9. Async Discipline

Even without background work, the concurrency model must be correct from day one.

1. **Every route handler, service, and repository function is `async def`** . A sync route handler is silently offloaded to a thread pool by FastAPI, which changes the concurrency model and defeats the async database driver.
 2. **Never block the event loop.** No `time.sleep` , no `requests` , no sync database driver.
Any unavoidable blocking call goes through `await asyncio.to_thread(...)` .
 3. **One `AsyncSession` per request**, provided by a `get_session` dependency, committed or rolled back exactly once, and never shared across tasks.
 4. A single `AsyncSession` cannot multiplex concurrent queries. If you reach for `asyncio.gather` over the same session you will get `InterfaceError: another operation is in progress` . Either use a session per concurrent call, or run sequentially.
-

10. API Specification

Global prefix `/api` . Declare path parameters with types (`user_id: int`) so FastAPI validates and coerces them at the boundary — manual `int(...)` casting inside a handler is a review failure, because a non-numeric path segment then surfaces as a 500 instead of a 400.

10.1 Health

GET /health **200**

Not wrapped in the standard envelope:

```
{ "status": "ok!", "timestamp": "2026-08-03T22:14:05.123Z" }
```

Must not touch the database — this is a liveness probe, and it should stay green while the database is down so a supervisor does not restart a healthy process over a dependency outage.

10.2 Users — /api/users

No `x-user-id` header; user creation is the bootstrap path.

GET /api/users 200

`data` : array of { `id`, `email`, `name`, `slug`, `timezone`, `createdAt`, `updatedAt` } .

GET /api/users/{user_id} 200

`data` : a single user. **404** if absent.

POST /api/users 201, message "User created successfully"

Field	Type	Required	Rules
<code>email</code>	<code>EmailStr</code>	yes	valid email
<code>name</code>	<code>str</code>	yes	1–100 chars
<code>slug</code>	<code>str</code> <code>None</code>	no	1–100 chars; derived from <code>name</code> when omitted
<code>timezone</code>	<code>str</code>	no	default "UTC" ; must resolve via <code>zoneinfo.ZoneInfo</code>

Behaviour: reject a duplicate email with **409** "Email already in use" . When `slug` is omitted, derive it from `name` ; on collision, append a numeric discriminator (`jane-doe-2`)

rather than failing the request.

Validate `timezone` against `zoneinfo` at the schema layer. An invalid IANA zone stored here

becomes an unrecoverable crash inside Part 2's slot generation, far from its cause.

PATCH /api/users/{user_id} 200, message "User updated successfully"

`email` , `name` , and `timezone` all optional, but **at least one must be present**. Enforce with a Pydantic `model_validator(mode="after")` returning 400 when the body is empty, so the rule appears in the schema rather than the service.

404 unknown user; **409** email already used by a different user.

DELETE /api/users/{user_id} 200, message "User deleted successfully"

`data` : the deleted user. Cascades remove that user's event types, availability rules, and exceptions.

10.3 Event Types — /api/event-types

Requires the `x-user-id` header. Scoped to that host.

GET /api/event-types 200

`data` : the host's event types, ordered `createdAt DESC` . Includes inactive ones — this is the host's management view, not the public listing.

GET /api/event-types/{event_type_id} 200

404 when unknown or owned by another host.

POST /api/event-types 201

Field	Type	Required	Default	Rules
<code>title</code>	<code>str</code>	yes	—	1–200 chars
<code>description</code>	<code>str</code> <code>None</code>	no	<code>null</code>	≤ 1000 chars
<code>durationMinutes</code>	<code>int</code>	no	30	15–120
<code>isActive</code>	<code>bool</code>	no	<code>true</code>	—
<code>locationType</code>	<code>Literal["online", "in-person"]</code>	no	<code>"online"</code>	—

locationValue	str None	no	null	—
bufferBeforeMinutes	int	no	0	0–120
bufferAfterMinutes	int	no	0	0–120
slug	str None	no	derived from title	^[a-z0-9-]+\$, 1–100 chars

Behaviour: derive the slug from `title` when omitted; **409** when `(hostId, slug)` already exists.

PATCH `/api/event-types/{event_type_id}` **200**

All create fields, all optional. **404**, and **409** when the new slug collides with another event type belonging to the same host.

DELETE `/api/event-types/{event_type_id}` **200**, `data: null`, `message` "Event type deleted successfully"

404 when unknown or owned by another host.

10.4 Availability — `/api/availability`

Requires the `x-user-id` header. All eight endpoints operate on that host's own rows.

Rules

Method	Path	Success
GET	<code>/api/availability/rules</code>	200 — array ordered by <code>weekday</code> , then <code>startTime</code>
POST	<code>/api/availability/rules</code>	201
PATCH	<code>/api/availability/rules/{rule_id}</code>	200
DELETE	<code>/api/availability/rules/{rule_id}</code>	200, <code>data: null</code>

POST body:

Field	Type	Required	Default	Rules
-------	------	----------	---------	-------

weekday	int	yes	—	0–6 (0 = Sunday)
startTime	str	yes	—	<code>^([01]\d 2[0-3]):[0-5]\d\$</code>
endTime	str	yes	—	same pattern; strictly after <code>startTime</code>
isActive	bool	no	true	—
timezone	str	no	"UTC"	must resolve via <code>zoneinfo.ZoneInfo</code>

`PATCH` accepts the same fields, all optional. **When a partial update supplies only one of**

`startTime` / `endTime`, merge it with the stored row before re-validating the ordering

invariant. Validating the payload in isolation lets `PATCH {"endTime": "08:00"}` invert a `09:00–17:00` rule and produce a row that can never yield availability. This is the single most

commonly missed requirement in this part.

Exceptions

Method	Path	Success
GET	<code>/api/availability/exceptions</code>	200 — ordered by <code>date</code>
POST	<code>/api/availability/exceptions</code>	201
PATCH	<code>/api/availability/exceptions/{exception_id}</code>	200
DELETE	<code>/api/availability/exceptions/{exception_id}</code>	200, <code>data: null</code>

POST body:

Field	Type	Required	Rules
<code>date</code>	str	yes	<code>YYYY-MM-DD</code>
<code>type</code>	<code>Literal[...]</code>	yes	<code>BLOCK_FULL_DAY</code> <code>BLOCK_PARTIAL</code> <code>ADD_AVAILABLE_WINDOW</code>
<code>startTime</code>	str None	conditional	required unless <code>type == "BLOCK_FULL_DAY"</code>
<code>endTime</code>	str None	conditional	same; must be after <code>startTime</code>

timezone	str	no	default "UTC"
reason	str None	no	≤ 500 chars

Model the conditional requirement as a **discriminated union** on `type`, or as a `model_validator`. Either way the constraint must be visible in the generated OpenAPI schema

rather than buried in a service. A `BLOCK_FULL_DAY` exception carrying times, or a `BLOCK_PARTIAL` without them, must be rejected at the boundary with 400.

Meaning of each type, for context — the behaviour is implemented in Part 2:

- `BLOCK_FULL_DAY` — the host is unavailable for the whole date, whatever the weekly rules say.
- `BLOCK_PARTIAL` — subtract this window from that date's availability.
- `ADD_AVAILABLE_WINDOW` — add this window on a date the weekly rules do not cover.

10.5 Public Event Discovery — `/api/public`

No header required. This is the invitee-facing surface.

GET `/api/public/users/{user_id}/event-types/{slug}` 200

Returns a deliberately narrow projection. Never serialise the model here:

```
{
  "success": true,
  "data": {
    "eventType": { "id": 1, "title": "30 Minute Meeting", "description": null,
                  "durationMinutes": 30, "locationType": "online" },
    "host": { "name": "Jane Doe", "email": "jane@example.com" }
  }
}
```

Return **404** with one indistinguishable message when the event type is missing, is inactive,

or the host does not exist. Distinct messages would turn this endpoint into an enumeration

oracle for hosts and their unpublished event types.

Note what is absent: available times. Part 2 adds the slot-listing endpoint that makes this page bookable.

11. Configuration

`pydantic-settings`, a single `Settings` class, loaded once at startup and injected as a dependency. **Fail fast and loudly on a missing required variable.** Defaulting

`DATABASE_URL`

to an empty string turns a config error into a confusing runtime failure minutes later.

Variable	Default	Purpose
<code>PORT</code>	<code>8000</code>	Uvicorn bind port
<code>ENVIRONMENT</code>	<code>development</code>	Gates traceback exposure in error details
<code>DATABASE_URL</code>	required	<code>postgresql+asyncpg://...</code>

Part 2 extends this table. `.env` must be gitignored; `.env.example` contains placeholders only.

12. Running Locally

You need a PostgreSQL 16 instance and nothing else. How you run it is your choice — a local install, a managed instance, or a container. A single `docker run` with a published port is the quickest route if you have Docker available.

Your `README.md` must state exactly which approach you used and give the commands to reproduce

it from a clean machine: start the database, apply `alembic upgrade head`, and run the API.